



Competitive pricing intelligence · Platform design

TVD OTA FareIQ

Knowing what the market charges, what a trip really costs our customer, and what we should price today.

150

routes monitored

4

data source tiers

07:00

daily review lands

100%

changes human approved

AGENDA

What we will cover

01 The morning question

What the platform exists to answer, and why today's process cannot.

02 Architecture and data

Seven stages, four source tiers, one warehouse.

03 Pricing methodology

True customer cost, and the test a price cut must pass.

04 Delivery

The Apps Script dashboards, the daily cadence, the roadmap and the risks.

THE QUESTION IT ANSWERS EVERY MORNING

What is the market charging today, where are we positioned, what price should we offer, why, and what revenue or margin impact should we expect?

Today that answer takes an analyst a morning of manual checking, covers a handful of routes, and leaves no record of why a price moved.

THE PROBLEM

Why this is worth building now

1 Manual and partial

Pricing checks cover the routes someone remembers to check, on the day they remember to check them.

2 Headline comparison

We compare advertised fares. Customers pay fares plus bags, seats and payment fees.

3 No margin view

A price is judged against a competitor, not against what the booking actually earns us.

4 No record

When a price moved, nobody can reconstruct what the market looked like at the time.

Each of these four is a solved problem in the design that follows.

SCOPE

Seven questions, answered before 07:00

- 1 Are we cheaper or more expensive than our competitors?
- 2 Which routes are overpriced, and which are underpriced?
- 3 Which competitor is genuinely the cheapest?
- 4 What does the trip really cost after taxes, baggage and fees?
- 5 What price should we offer today, and why that price?
- 6 Where can we raise margin without losing the sale?
- 7 Which changes need attention this morning, ranked by value?

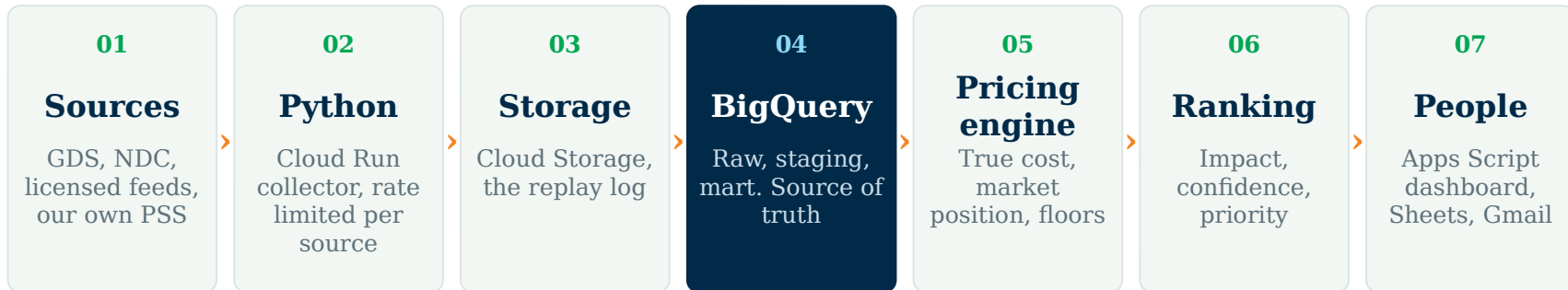
Not new questions. The existing ones, answered daily and with the evidence attached.

01

Architecture and data

Seven stages, four source tiers, one warehouse of record. Every stage replayable from the one before it.

How a fare becomes a recommendation



Why we land every response to Cloud Storage before BigQuery

- A load job is free where streaming inserts are not.
- A failed load leaves the file intact for a rerun, rather than a hole in the day.
- When a parser bug is found in March, January's data can be reprocessed rather than written off.

Built on what we already run

● Collect and process

- Python 3.11, async collectors
- Cloud Run, one service per job
- Cloud Scheduler, Africa/Lagos times
- Cloud Storage for raw JSONL

● Store and analyse

- BigQuery: raw, staging, mart, ML
- Partitioned and clustered by route
- BigQuery ML for demand and elasticity
- pandas and scikit-learn for mining

● Operate and control

- Apps Script web app: six dashboards
- Google Sheets for pricing rules
- Gmail for the 07:00 digest
- Secret Manager, IAM, GitHub Actions

DATA SOURCES

Four tiers, in strict preference order

Our own PSS

Tier 1

The only source giving supplier cost beside selling price. Every margin figure starts here.

1.00
trust

GDS and NDC

Tier 2

Amadeus for breadth, NDC for branded fares and ancillary prices in the same response.

0.95
trust

Licensed feed

Tier 3

Competitor prices under contract. The tier that legitimately answers what the market charges.

0.85
trust

Permitted public

Tier 4

Off by default. Reviewed terms only, six calls a minute, never the sole basis for a change.

0.60
trust

Trust score is not decoration: it follows the data into the confidence attached to every recommendation.

Reading the history, not just collecting it

● Scraping, where it is permitted

- A source is fetched only if the register lists it and its terms have been reviewed.
- A review older than ninety days stops collection by itself.
- robots.txt obeyed per path; its crawl delay beats our own rate limit.
- No CAPTCHA solving, no logins, no IP rotation. A 403 ends it.

● What the nightly job finds

- Prices far from their own cell's norm, separating a mistake fare from an outlier.
- Competitors measured on position, so a market-wide move flags nobody.
- A sustained reprice, told apart from a one-day blip.
- A seller we usually see and did not see today.

Two patterns worth money: the departure day of the week the market charges more for, and the days-to-departure band where it steps up. Our markup is flat across both today, so each one is margin on one side and share on the other.

Compliance built in as a control

The source register is executable

The public-source collector reads the register at runtime and refuses to run against anything without a current legal review. A review older than ninety days stops collection automatically, with nobody needing to remember.

Every row carries its legal basis

Contract, licence, own system or permitted public is recorded on each observation. Any figure on any dashboard traces back to the right under which we obtained it, which is what lets us share a report at all.

Standing rules, enforced in code rather than in a policy document

- robots.txt is parsed and obeyed per path; its crawl delay overrides our own rate limit when slower.
- The user agent identifies us honestly with a contact address. Conditional requests never re-fetch an unchanged page.
- No CAPTCHA solving, no logins, no session replay, no IP rotation. A 401 or 403 ends collection rather than starting a workaround.
- Never let a public source alone justify a price change.

One row means exactly one thing

TABLE	GRAIN: ONE ROW PER	WHY IT EXISTS
fact_offer	One priced offer, per seller, per hour	Adds true customer cost and FX at the observation date
fact_market_snapshot	Route x date x cabin x hour	Market statistics. What dashboards and the engine both read
fact_price_recommendation	One recommendation	Every recommendation ever made, acted on or not
fact_price_decision	One human decision	Append only. Who priced what, when, on what evidence
fact_booking	One ticketed booking	One agreed margin definition, used everywhere
dim_airline_fee	One fee version	Type 2. The fee history is the product, not a side effect

Get the grain wrong and every number downstream is quietly wrong. It is stated on every table in the design.

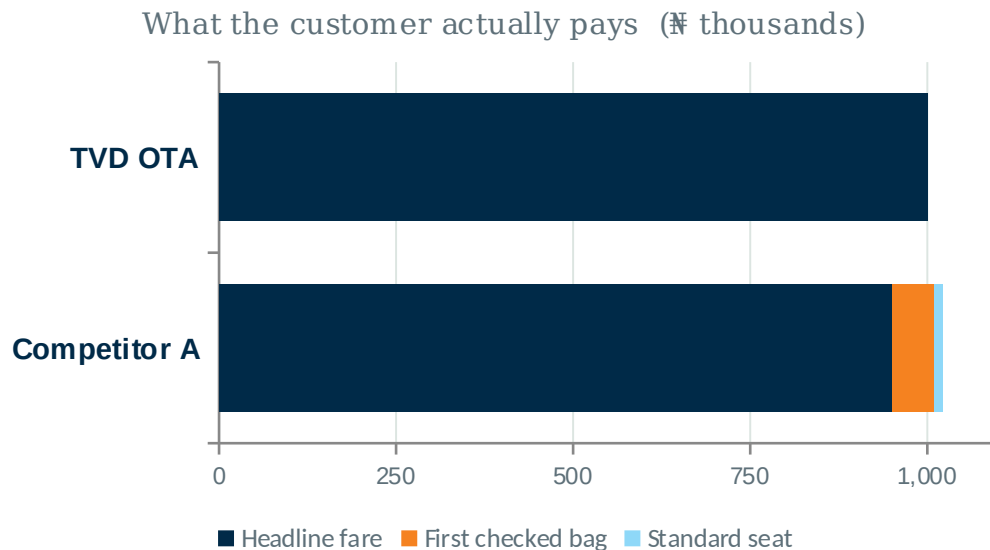
02

Pricing methodology

Two ideas do most of the work. One stops us comparing the wrong numbers. One stops us cutting price for the wrong reason.

IDEA ONE · TRUE CUSTOMER COST

Cheaper on screen is not cheaper



Competitor A: ₹950,000 fare + ₹60,000 bag + ₹12,000 seat = ₹1,022,000.

TVD OTA: ₹1,000,000, bag and seat included.

On the headline

₹50,000 dearer

All in

₹22,000 cheaper

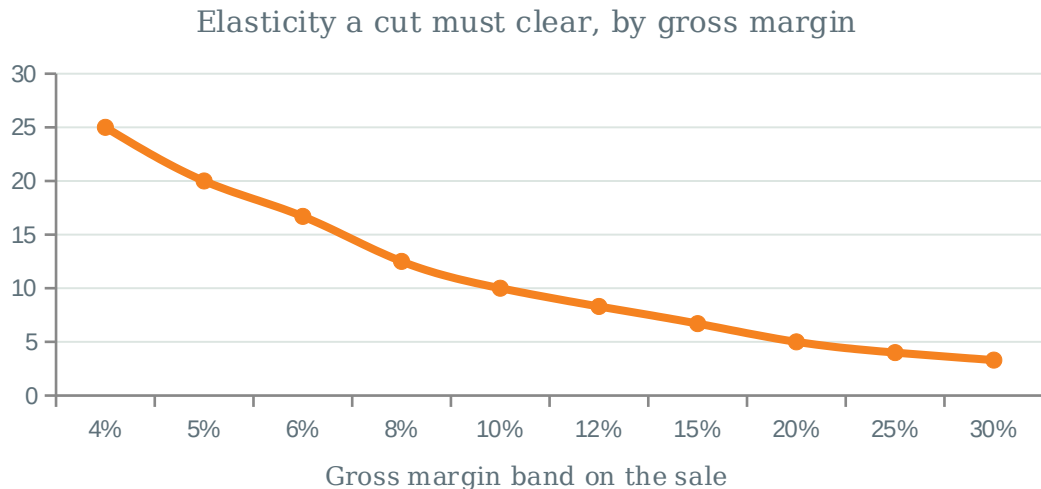
A platform comparing headline prices discounts here, gives away margin to fix a problem that does not exist, and repeats it every day until somebody notices.

Every comparison in the platform runs on this figure, never on the advertised fare.

IDEA TWO • THE VALUE GATE

A cut has to pay for itself

Margin is unchanged by a price cut only when $|elasticity| = price \div (price - cost)$. Below that bar, discounting buys volume at a loss.



Read it this way

- At a 20% margin, a cut pays if a 1% price drop wins 5% more share. Flight shopping clears that.
- At a 5% margin it must win 20% more share. It will not.
- Below roughly 7% margin, no plausible elasticity makes a cut pay.

Where the gate fails, the engine escalates instead of discounting: the gap is in the supplier cost, not in the price.

A DISTINCTION WORTH THE MINUTE

The parameter that decides everything

Market demand elasticity

-1.0 to -1.7

How total travel demand on a route responds to the price level. This is what an airline uses to set fares.

Wrong tool for us.

Competitive share elasticity

-4 to -15

How our share of a fixed pool of shoppers responds to our position on the same screen as everyone else.

This is the one that governs an OTA pricing decision.

Flights are close to a commodity and metasearch sorts by price, so share moves violently. Using the airline's number where ours belongs makes the break-even test unreachable at any OTA margin, and the engine would never recommend a cut at all. We found this while building, and fixed it.

TAXONOMY

Five classifications

COMPETITIVE

index 0.97 to 1.03

At market on true cost. No action, and no daily churn to prove the system is working.

WATCH

index 0.93 to 1.08

Drifting. Worth a look when the gap is large in money, not merely in percent.

UNCOMPETITIVE

outside those bands

A real gap in either direction. Acted on only after the value gate.

MARGIN OPPORTUNITY

index below 0.93

We are leaving money on the table. Usually the most valuable column, and the one teams forget to build.

PROTECTED

at the floor, or gate failed

No mechanical answer. Renegotiate, switch carrier, or accept the position. Never a silent hold.

Eight steps, and the guards come first

- 1 Guard**
Is this cell safe to act on at all? Fewer than three competitors, stale data or an outlier cheapest price and the engine declines to have an opinion.
- 2 Boundaries**
Minimum price from cost plus the binding floor, whichever of the percentage or absolute floor is higher.
- 3 Target**
Market median and cheapest comparable, blended by how tight the market is, then adjusted for our fee position.
- 4 Constrain**
Floor, daily move cap, rounding to a publishable price, and suppression of moves too small to be worth the churn.
- 5 Classify**
One of the five classes on the previous slide.
- 6 Quantify**
Volume response, damped to seventy percent of the modelled effect, then revenue and margin impact.
- 7 Value gate**
If a cut destroys margin it is not a recommendation. Escalate, and name the break-even elasticity.
- 8 Score and explain**
Confidence from six weighted components, priority from value times confidence, and a rationale in plain English.

Steps 1 and 7, in orange, are the two that stop the engine doing something expensive.

WORKED OUTPUT

The engine on seven real cells

SCENARIO	CURRENT	FLOOR	NEW PRICE	CHG %	CLASS	ACTION
Overpriced, healthy margin	1,150,000	880,829	1,058,000	-8.0%	UNCOMPETITIVE	DECREASE
Overpriced, razor margin	1,150,000	1,119,171	1,150,000	—	PROTECTED	ESCALATE
Underpriced, margin room	880,000	808,290	940,600	+6.9%	MARGIN OPP	INCREASE
Bag advantage vs cheapest	1,050,000	808,290	975,000	-7.1%	WATCH	DECREASE
Competitor below our cost	1,050,000	932,642	1,050,000	—	PROTECTED	ESCALATE
Only one competitor seen	1,050,000	932,642	1,050,000	—	WATCH	INVESTIGATE
Outlier cheapest ignored	1,050,000	932,642	1,050,000	—	WATCH	INVESTIGATE

Four of the seven produce no price change, and that is the point. A tool that recommends a move on every row is reacting to the market rather than reading it, and the analyst learns to ignore it within a fortnight.

Recommend, then approve

What every decision records

- Price before, recommended and approved
- Who approved it, and exactly when
- A frozen copy of the market at that moment
- Expected margin and confidence
- The actual outcome at T+7 and T+30

What a person may not override

An analyst may overrule the engine on any price. An analyst may not approve a price below the margin floor: the service refuses it and names the rule. That argument belongs in a versioned, reviewed pricing rule, not in one booking at a time.

Confidence, and what it is for

0.25

Coverage

0.20

Data quality

0.20

Fee certainty

0.15

Freshness

0.10

Model

0.10

Stability

Coverage carries the most weight because it is the assumption most often violated and least often noticed. Confidence gates the priority band, and later it will gate any automation.

03

Delivery

The six dashboards, when the team sees them, how it gets built, and what would make it fail.

Six dashboards, decision on the row

1 Overview

Price index against the market, with its healthy band, readable in three seconds. Plus a coverage gauge, so we see when the numbers are not trustworthy.

2 Daily review

The working page. Approve, modify or reject in the table itself. A reason is mandatory on every override.

3 Competitors

Who wins on price, how often, whose strategy is changing, and what the mining job flagged overnight.

4 Airline fees

The fee matrix the commercial team will print and keep. Analyst and lead only.

5 Routes

Price and demand on a shared date axis, so a price move can be judged against what it did to volume.

6 Opportunities & audit

Where the money is, and how well the engine's forecasts have actually held up.

Why not a BI report

	APPS SCRIPT WEB APP	A BI REPORT
Approve, modify, reject in place	Yes, and it is the point	No, read only
Access control	Workspace groups, three roles, per request	Report sharing only
Enforces the margin floor	In the dialog and again in the service	No
Query cost control	Byte cap and shared cache on every query	Per viewer, harder to bound
Says when data is stale	A banner that stops pretending	A quiet timestamp
Licence cost	Included in Workspace	BI Engine is extra

The deciding row is the first one. Putting the decision two clicks from the evidence separates a dashboard the team uses every morning from one they open in week one and never again.

AUTOMATION

The day, in Africa/Lagos time

Hourly

Sweep the 40 highest revenue routes

Every 4h

Sweep the secondary routes

03:00

Sweep the long tail

05:30

Build offers and market snapshots

06:00

Refresh the models, weekdays only

06:15

Write the day's recommendations

06:45

Populate the Daily Review sheet

07:00

Digest lands in the team's inbox

Daytime

Analysts approve, modify or reject

18:00

Measure outcomes at T+7 and T+30

Approvals happen in Google Sheets, where the team already works. A workflow people avoid is worse than none.

IMPLEMENTATION ROADMAP

Something useful by week six

0	Wks 1-2	Foundation	GCP, IAM, CI. Land our own transactions and agree one margin definition with Finance.
1	Wks 3-6	Collection	Collector service, GDS and NDC, 40 routes. Dashboard shell live.
2	Wks 7-10	True cost	Licensed competitor feed, fee catalogue, true customer cost live.
3	Wks 11-15	The engine	Rules in Sheets, recommendations, in-app approvals, audit trail, mining.
4	Wks 16-22	Learning	Outcome measurement, route elasticity, demand forecast, deliberate price tests.
5	Wks 23-30	Earned automation	Small changes auto-applied on proven routes. Scale to 150 routes.

Phase 0 asks whether supplier cost is reliable per booking. Week two is a far better time to learn that than week fourteen.

RISKS, AND THE ANSWER ALREADY DESIGNED IN

What would make this fail

RISK	HOW IT SHOWS UP	WHAT ALREADY ANSWERS IT
Supplier cost unreliable	Margin figures Finance will not sign	Phase 0 exists to find this first
Thin competitor panel	Confident advice from two data points	Hard three-seller minimum, coverage score
Ancillary data stays sparse	True cost quietly reverts to headline	Fee confidence demotes the recommendation
Elasticity from thin data	Revenue figures nobody should believe	Shrinkage to a regional prior, source always shown
Analysts ignore the tool	Sheet untouched by week three	Approval where they already work, sorted by money
A licensed feed changes terms	A commercial dispute nobody saw	Legal basis on every row, register reviewed quarterly

None of these is hypothetical. Each one has been designed against rather than noted and forgotten.

WHAT WE ARE ASKING FOR

Approve phase 0

- 1 **Two weeks** to land our own transaction data and prove supplier cost is reliable
- 2 **One decision** Finance and Commercial agree a single margin definition, in writing
- 3 **One owner** a named pricing analyst who will work the daily review from phase 3

The design, the BigQuery model, the pricing engine and the approval workflow are already written and tested. What phase 0 buys is the confidence that our own data can carry them.